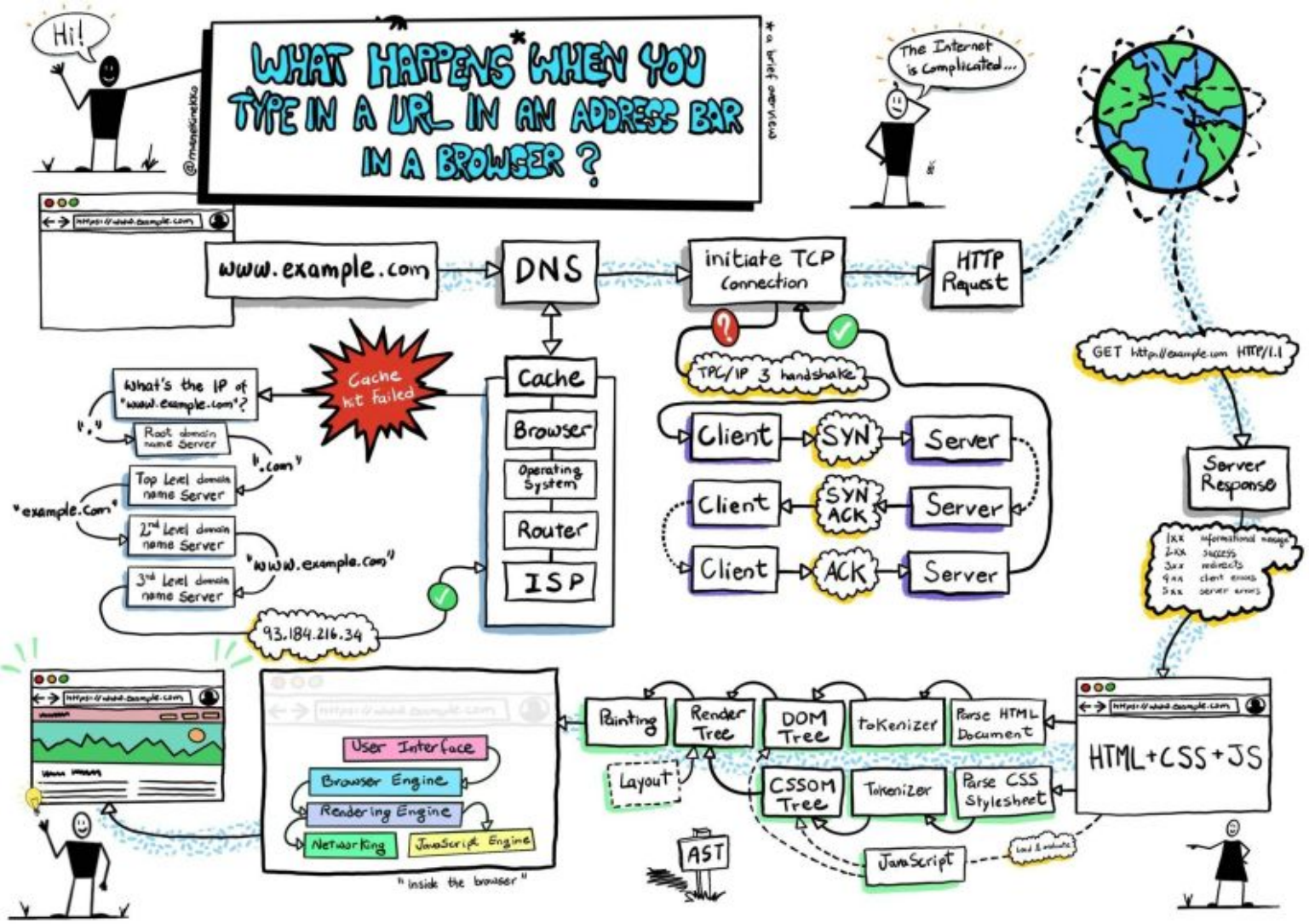
Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	**REST-Services**	**134**
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343



Was fehlt noch?

WIE baut man **gute** Services?

REST



- **Re**presentational **S**tate **T**ransfer
- Beschrieben 2000 in der Dissertation von Roy Fielding*
- Basiert auf 6 „Constraints“:

1. **Client-Server**
2. **Zustandslos**
3. **Caching**
4. **Einheitliche Schnittstelle**
5. **Layered Systeme**
6. **Code-on-Demand (optional)**

* https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf

REST Constraints 1-3

1. **Client-Server**
   Dient der Entkopplung: Client weiß nichts darüber, wie die Daten gespeichert sind.

2. **Zustandslos**
   Jeder Aufruf des Clients muss alle Informationen enthalten, die der Server braucht, um den Aufruf zu verstehen.
   Jeglicher Zustand wird im Client gespeichert – kein Session-Kontext im Server!

3. **Caching**
   Antworten werden als cacheable oder non-cacheable gekennzeichnet.
   Ist eine Antwort cacheable, dann kann der Client entscheiden, ob er diese Antwort später wiederverwendet.

4. **Einheitliche Schnittstelle**
   Jede Ressource bekommt eine URI.
   Repräsentationen zur Änderung von Ressourcen.
   Selbsterklärende Nachrichten mit Verben (i.W. GET, POST, PUT).

5. **Layered Systeme**
   Komponenten „sehen“ nur den nächsten Layer.

6. Optional: **Code-on-Demand**
   Bei Bedarf kann Code vom Server zum Client geschickt werden.

Ressourcen

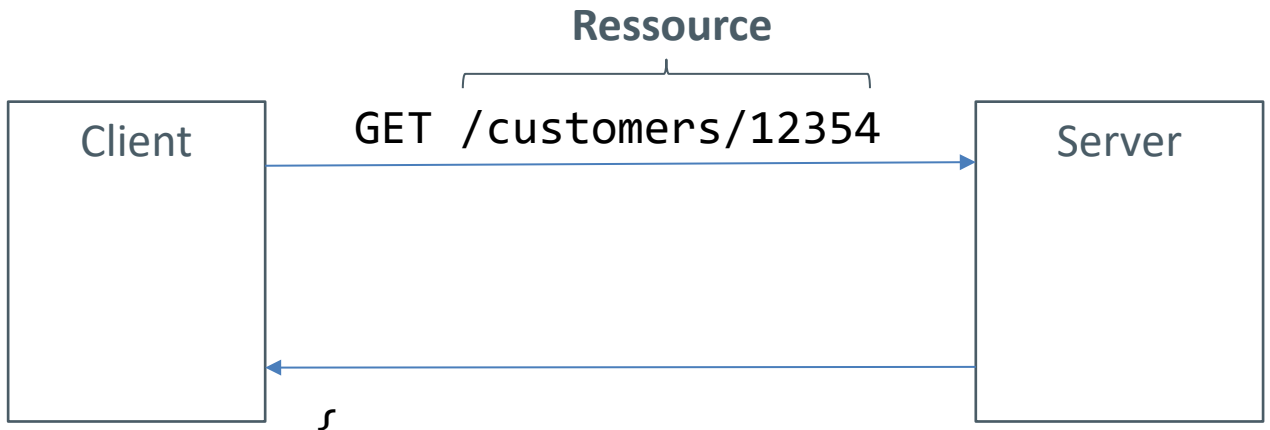
In REST geschieht der Datenzugriff immer über “**Ressourcen**”:

- Inhaltlich zusammengehörende Einheiten wie z.B. “Kunde”, “Liste von Kunden”, “Produkt”, …
- Werden eindeutig über eine **URI** (Uniform Resource Identifier) identifiziert
  Beispiele:
  - https://www.myshop.com/products/42
  - https://www.myshop.com/customers/4711

- Ressourcen können geschachtelt sein:
  - http://www.myshop.com/customers/4711/order/5
    (der 5. Auftrag des Kunden mit der Nummer 4711)

- Wenn eine Ressource eine Liste von Ressourcen darstellt, kann sie abgefragt werden:
  - http://www.myshop.com/customers?search=Joe%20Miller

Was ist daran „representational“ ?




```
{
  “vorname” : “Max”,
  “nachname” : “Mustermann”,
  “gebdatum” : “1.1.1970”
}
```

Repräsentation des aktuellen Zustands der Ressource
(hier in JSON). Könnte aber auch XML, HTML, Text, pdf , … sein

Ressource ≠ Repräsentation !!!



Best Practices

- Ressourcen sind Nomen möglichst im Plural
  Gut: `/customers/1234` Schlecht: `/getCustomers/1234`

- Ohne die Angabe einer ID werden alle Entitäten angezeigt
  `/customers` gibt also die Liste aller Kunden zurück (mit nur wenigen Details).

- Verschachtelung sparsam verwenden
  Schlecht: `/companies/123/departments/1/users/4711/orders`
  Besser: `/orders?userId=4711`

- Ressourcen sind nicht zu groß und nicht zu klein
  Warum?

HTTP Statuscodes

Generell gilt folgende Aufteilung:

100-199	200-299	300-399	400-499	500-599
Informationen (sehr selten benutzt)	Erfolgreich z.B. 200 OK	Umleitungen z.B. 301 (moved permanently)	Client Fehler z.B. 400 (bad request)	Server Fehler z.B. 500 (Internal Server Error)

Komplette Liste: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Wichtig ist, dass Sie bei einem Fehler eine korrekte und für den Nutzer sinnvolle Fehlermeldung zurückgeben!

HTTP Verben richtig benutzen

- Ressourcen kann man anlegen (**Create**), lesen (**Read**), ändern (**Update**), löschen (**Delete**).
- Diese entsprechen (in etwa) den HTTP-Verben:

	POST	GET	PUT	DELETE
`/products`	Erstellt ein neues Produkt	Holt Liste aller Produkte	Erlaubt mehrere Produkte gleichzeitig anzulegen (unüblich)	Löscht alle Produkte (Vorsicht!!!)
`/products/123`	Fehler	Details des Produkts 123	Ändert Produkt 123, wenn es existiert	Löscht Produkt 123
`/product/123/vendors`	Erstellt neuen Anbieter für Produkt 123	Holt Liste aller Anbieter des Produkts 123	Erlaubt mehrere Anbieter für Produkt 123 gleichzeitig anzulegen (unüblich)	Lösche alle Anbieter von Produkt 123 (Vorsicht!!!)

- POST legt neu an; PUT ändert (wenn vorhanden) den ganzen Datensatz.
- PATCH kann einzelne Attribute ändern

Empfohlene Statuscodes

`GET`	Ressource gefunden: 200 (OK)	Sonst: 404 (Not found)		
`POST`	Anlegen erfolgreich: 201 (Created) und Location zurückgeben z.B. /users/78658	Request erfolgreich, aber nichts angelegt: 204 (No Content) z.B. /logout	Daten des Clients falsch: 400 (Bad Request)	
`PATCH`	Änderung erfolgreich, aktualisierte Ressource zurückgeben: 200 (OK)	Änderung erfolgreich, keine Ressource zurückgeben: 204 (No Content)		
`PUT`	Anlegen erfolgreich, Ressource zurückgeben: 200 (OK)	Anlegen erfolgreich, keine Ressource zurückgeben: 204 (No Content)	Daten des Clients falsch: 400 (Bad Request)	Anlegen nicht möglich: 409 (Conflict)
`DELETE`	Löschen erfolgreich: 204 (No Content)	Sonst: 404 (Not found)		

Komplette Liste: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Filtern und Paginieren

- Oft sind die Daten nur zum Ansehen gedacht. Dann macht es keinen Sinn, dem Browser Hunderte von Einträgen zu senden.
- Besser:
  `/orders?limit=25&offset=50`
- D.h. maximal 25 Einträge zurückgeben, beginnend mit dem 50-ten.

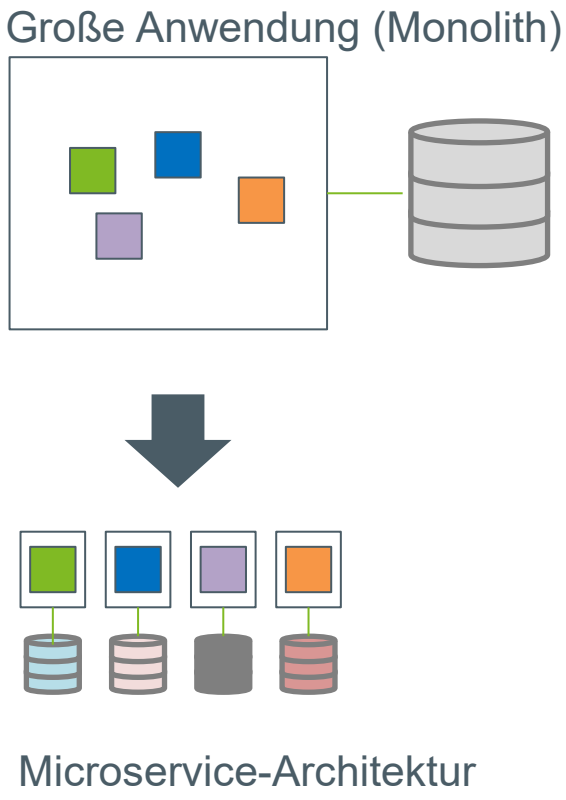


MicroServices === RESTService?

“Microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API” – Martin Fowler (https://martinfowler.com/microservices/)

Microservices sind also mehr eine Vorgehensweise, ein Muster dafür, wie man aus vielen kleinen Services, die REST Services sein können, eine Anwendung zusammenbaut.

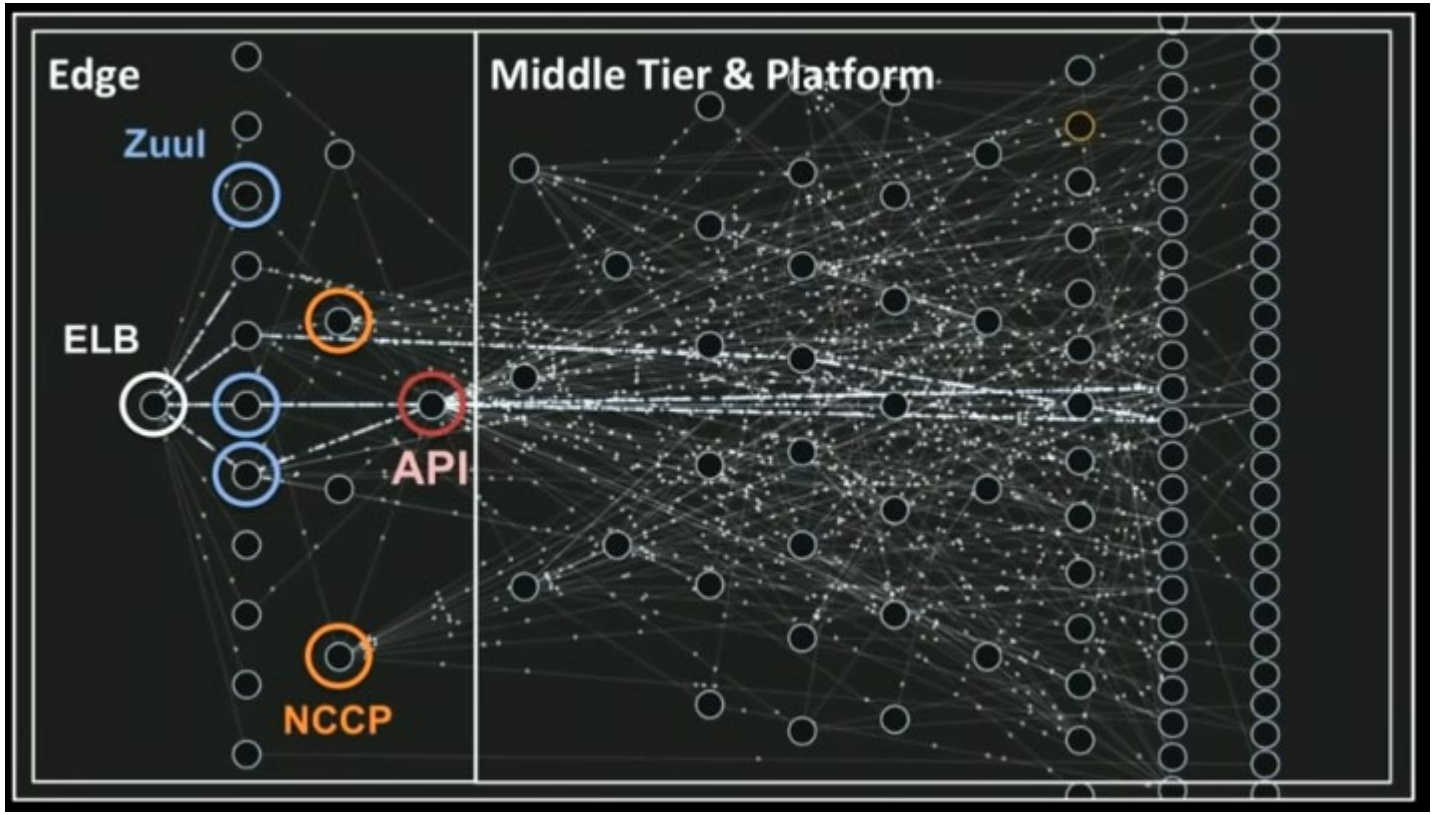
Jeder Service erfüllt eine fachliche Aufgabe, läuft auf einem eigenen (virtuellen) Server und hat seine eigene Datenbank.



Microservice-Architektur

MicroServices - Beispiel

Netflix benutzt ein Geflecht von Tausenden von Microservices, die miteinander kooperieren, um die Anfragen der 200 Mio. Benutzer zu beantworten (OHNE das eigentliche Streaming).



Mehr Kekse!

- HTTP ist ein zustandloses (stateless) Protokoll
- Viele Websites scheinen den Benutzer aber „wiederzuerkennen“
- Websites können Cookies (siehe PI1) mit dem HTTP-Request an den Browser schicken, wo sie (meistens) abgespeichert werden.
- Wie kann man einen Cookie setzen und abfragen?

```
// Setzen
function setCookie(name:String, expiryDays:number =7,
                   path:String="/") {
    let expires:String = "";
    let date = new Date();
    date.setTime(date.getTime() + (expiryDays * 24 * 60 * 60 *
1000));
    expires = "; expires=" + date.toUTCString();
   const cookieString = "MeinKeks" + "=" + name + expires + ";
                        path=" + path;
   document.cookie= cookieString;
}
```

```
// Abfragen
// Liste aller Cookies
const cookies: String[] = document.cookie.split(';');

 cookies.forEach(cookie => {
       const trimmedCookie = cookie.trim();
       const [name, value] = trimmedCookie.split('=');
       console.log(`${name}: ${value}`);
});
```

Mehr unter: https://developer.mozilla.org/en-US/docs/Web/API/Document/cookie

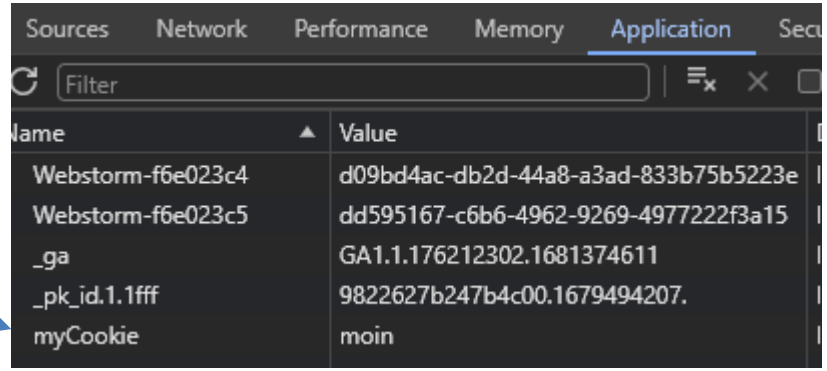
Server-side Cookies

- Auf der vorigen Seite wurde der Cookie durch JavaScript gesetzt. Das kann man für unkritische Inhalte wie z.B. User Preferences tun.
- Cookies, die z.B. dem Session-Management dienen (siehe nächste Seite), sollte man im Server generieren und in der HTTP-Antwort senden, z.B. mit vertX:

```
router.get("/blabla").handler(this::something);
...
private void something(RoutingContext routingContext) {
  HttpServerResponse response = routingContext.response();

  // Set a cookie with name "myCookie " and value „moin"  to expire in 1 hour
  response.putHeader ("Set-Cookie" , "myCookie =moin; Path=/; Max -Age=3600" );

  // Send the result as a JSON response
  routingContext.response()
    .putHeader("content-type", "application/json")
    .setStatusCode(200)
    .end(Json.encodePrettily(new JsonObject().put("Greeting", "Moin!")));
}
```



Cookie-Abfrage im Vert.x Server

```
package de.thm.mni.cookies;

import io.vertx.core.Vertx;
import io.vertx.core.http.Cookie;
import io.vertx.core.http.HttpServerRequest;
import io.vertx.core.http.HttpServerResponse;
import io.vertx.ext.web.Router;
import io.vertx.ext.web.RoutingContext;

public class InspectCookie {

  public static void main(String[] args) {
    Vertx vertx = Vertx.vertx();
    Router router = Router.router(vertx);


router.route("/inspectCookie").handler(InspectCookie::inspectCookieHandler);

    vertx.createHttpServer().requestHandler(router).listen(8080);
  }
```

```
private static void inspectCookieHandler(RoutingContext routingContext) {
    System.out.println("In inspectCookieHandler");
    HttpServerRequest request = routingContext.request();
    HttpServerResponse response = routingContext.response();


    Cookie cookie = request.getCookie("myCookie");

    if (cookie != null) {
      String cookieName = cookie.getName();
      String cookieValue = cookie.getValue();
      String cookiePath = cookie.getPath();
      long cookieMaxAge = cookie.getMaxAge();
      String ausgabe = "Name: "+cookieName
                     + ", Value = " + cookieValue
                     + ", Path = " + cookiePath
                     + ", MaxAge = " + cookieMaxAge;
      response.end("Cookie value: " + ausgabe);
    }else {
      response.end("No cookie found");
    }
  }
}
```

Session-Management

- Eine Session ist eine Abfolge von HTTP Anfragen und Antworten
- HTTP kennt keine Sessions, aber
  - Der Server kann Daten in einem Cookie auf dem Client speichern.
    Der Cookie enthält eine eindeutige **Session-ID**.
    Mit jeder Anfrage schickt der Client den Cookie.
  - Der Server speichert zusätzlich die Session-Daten auf dem Server (1 Session pro Client).
    Durch die Session-ID im Cookie kann der Server die richtige Session für den Client finden.



Das Session-Objekt

- Das Interface `Session` bietet Methoden, um die Informationen einer Session abzufragen und zu setzen.
- Sessions werden gelöscht, wenn der Browser geschlossen wird, und arbeiten mit einer UUID („Unique Universal Identifier“, einer 128 bit Zahl).

```
private void greeting(RoutingContext routingContext) {
  String name="";
  Session session = routingContext.session();

  // check whether there is already information in the session
  if (session.data().containsKey("user"))  name = session.get("user");
  else session.put("user", "Uwe Meyer");
  }

  // Send the result as a JSON response
  routingContext.response()
    .putHeader("content-type", "application/json")
    .setStatusCode(200)
    .end(Json.encodePrettily(new JsonObject().put("Ausgabe:", "Hallo " + name
+ "!")));
}
```

Session initialisieren

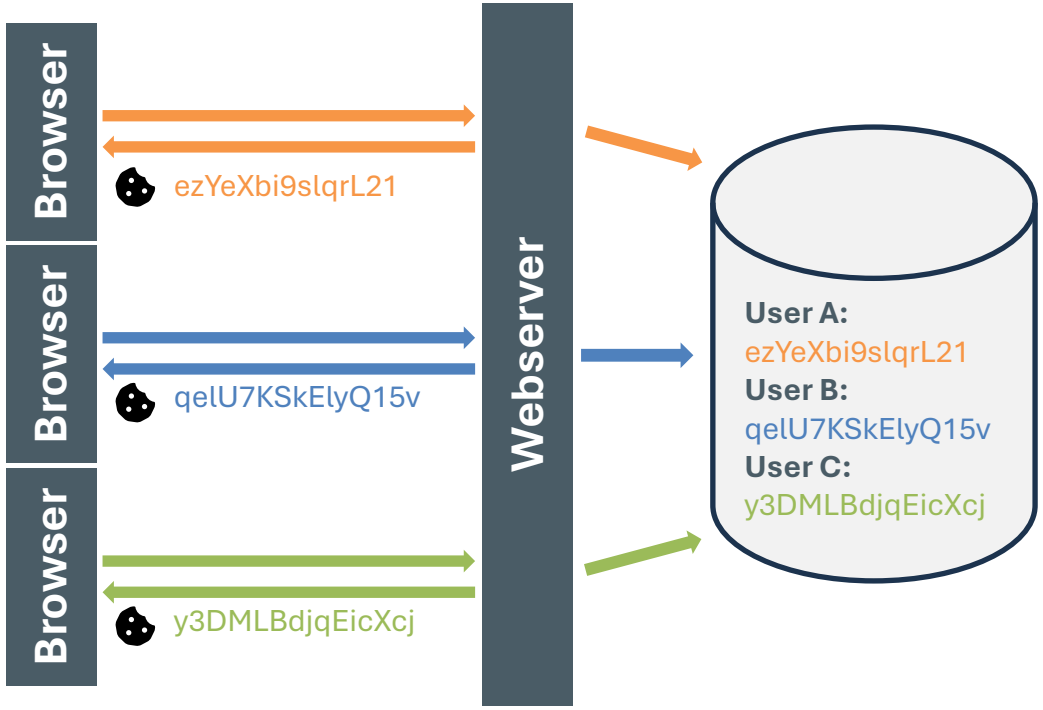
Session abfragen

Daten in Session speichern

Probleme von Session-Cookies

Speicherverbrauch der Session-IDs

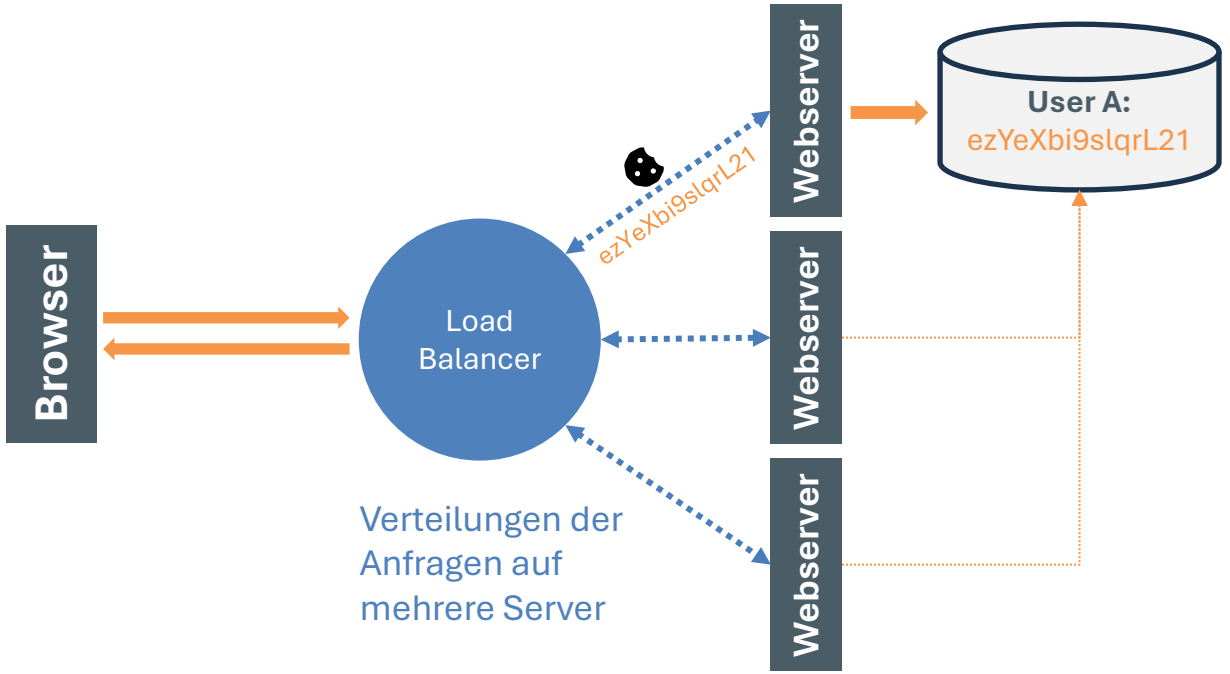
Jede Session-ID muss auf dem Server permanent gespeichert werden (z.B. in einer Datenbank)
→ Speicherverbrauch pro Client



Systeme mit mehreren Servern

Jeder Server muss in der gleichen Datenbank nachsehen, ob die Session gültig ist

→ Engpass in der Skalierbarkeit



JSON Web Tokens (JWTs, gesprochen 'jot')

…sind kompakte, URL-sichere Tokens, die Informationen zwischen zwei Parteien in Form von JSON-Objekten sicher und verifiziert übertragen, oft verwendet zur Authentifizierung und Autorisierung.

Bestandteile der Tokens:

- Der erste Teil ist der Header (Typ und genutzte Algorithmen)
- Der zweite Teil ist die Payload (Nutzdaten)
- Der dritte Teil ist die Signatur

Beispiel:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MiIsIm5hbWUiOiJBcnRodXIiLCJhZG1pbiI6dH
J1ZX0.6vxIu9l2sdctwx0XOYKv0FVNaanY28GH6anpSEhz1wY
```

Alle Teile sind **Base64Url-kodiert**, was sicherstellt, dass sie in URLs verwendet werden können, ohne dass sie umkodiert oder verändert werden müssen (= URL-sicher). Das ist KEINE Verschlüsselung!

(Base64 benutzt nur 64 Zeichen: [a-z], [A-Z], [0-9], +, / und =, siehe https://de.wikipedia.org/wiki/Base64)

Aufbau eines JWTs

❶ Header

JSON-Element beschreibt den Token-Typ und Signaturmethode

```
{
    "alg": "HS256",
    "typ": "JWT"
}
```

❷ Payload

JSON-Element beschreibt die Claims (Datenfelder)

```
{
    "sub“  : “ucmy47",
    "name“ : “Uwe Meyer",
    "admin": true
}
```

❸ Signature

Genormte Signatur mit einem nur dem Server bekannten Geheimnis

```
var encodedString =
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload);
var hash =
  HMACSHA256(encodedString, secret);
```

var jwt = base64UrlEncode(header) + "." + base64UrlEncode(payload) + "." + base64UrlEncode(hash)

(Anmerkung: Einige JSON-Felder sind in der JWT-Signatur reserviert, siehe https://de.wikipedia.org/wiki/JSON_Web_Token)

Reservierte Felder in der Payload (Claims)

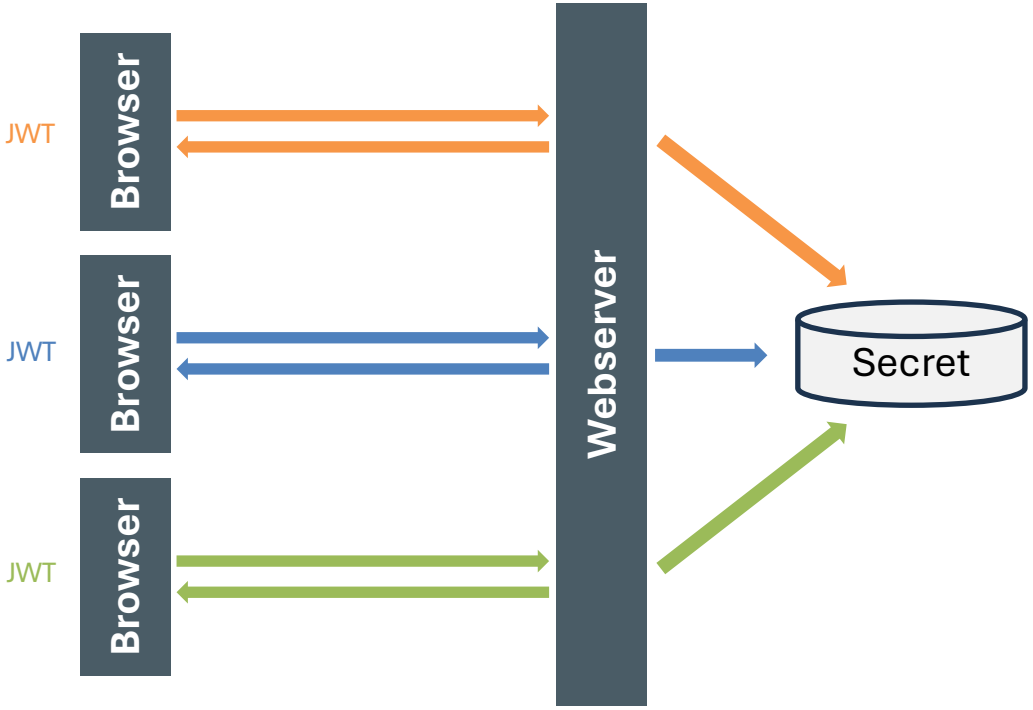
Feld	Name	Beschreibung
`iss`	Issuer	Der Aussteller des Tokens
`sub`	Subject	Definiert für welches Subjekt die Claims gelten. Das sub-Feld definiert also für wen oder was die Claims getätigt werden.
`aud`	Audience	Die Zieldomäne, für die das Token ausgestellt wurde.
`exp`	Expiration Time	Das Ablaufdatum des Tokens in Unixzeit. Eine Unixzeit ist die Anzahl der Sekunden seit 1970-01-01T00:00:00Z.
`nbf`	Not Before	Die Unixzeit, ab der das Token gültig ist.
`iat`	Issued At	Die Unixzeit, zu der das Token ausgestellt wurde.
`jti`	JWT ID	Eine eindeutige case-sensitive Zeichenfolge, welche das Token eindeutig identifiziert. Hiermit kann verhindert werden, dass das Token repliziert wird. Zum Beispiel eine durchgezählte Nummer, einen GUID oder ein Hashwert.

[Quelle: https://de.wikipedia.org/wiki/JSON_Web_Token#Payload]

Probleme gelöst?

Speicherverbrauch von JWTs

JWTs werden im Client gespeichert. Server muss sich nur ein Geheimnis (secret) merken



JTWs bei Systemen mit mehreren Servern

Jeder Webserver kennt das Secret und kann Anfragen aller Webserver verifizieren

